

Multi-threading optimization for TSP

TODO slide page, page headers Authors: Olivia Manz and Samuel Roland PCM Course - 2025

Contents

Introduction	2
Data structures	2
TSPParraTask & ParallelTaskRunner	2
Stop management	2
PriorityStack	2
LockFreeQueue	3
Work-stealing deque	3
Memory ordering challenges	5
Integration	6
Memory model	6
Crashes	7
Measures	7
Benchmarking system	8
Speedup	9
Efficiency	11
Perspectives	12
Conclusion	12
Bibliography	12

Introduction

The goal of this project is to optimize the repartition of work among a large numbers of threads. The TSP has been chosen for this challenge, with a starting code implementing an existing single-threaded resolution. We had access to a 256 cores and 126GB of RAM machine remotely to test our program. The core of the work is on the internal lock-free data structures, that enable an optimized work repartition and synchronisation.

The main ideas were to implement a lock free queue to replace the existing stack. This implementation also required to migrate part of the code to be safe in a multithreaded context. To further optimize the first results, we search for dedicated structures for this kind of problem and found the work-stealing deque, with the paper "Dynamic circular work-stealing deque" [1]. We measured the speedup and efficiency of the final version. Along the way we measured several combinations of the 3 available parameters: number of cities, number of thread and cutoff.

TODO: résumé des résultats sans chiffres précis

Data structures

In this section we present the structure of 2 core datastructures and one secondary structure also used. The first part is developed inside `parallel_work.hpp`.

TSPParraTask & ParallelTaskRunner

Stop management

To manage the exit of threads properly, all threads are checking a global counter to detect the end of the problem. For this, we initialize a global `std::atomic<uint64_t> _tasks_done;` to zero and a constant counter `uint64_t _total_todo_tasks_counter;` to the total amount of tasks to cover. To calculate how much tasks a tree or subtree contains, we precalculated at compile-time the amount of nodes in any tree for 0 to 25 cities in a global constant

`SUBTREE_NODES_COUNT_BY_TREE_HEIGHT`. As an example, the tree with 3 cities is composed of 5 nodes (level 0: 1 node, level 1: 2 nodes, level 2: 2 nodes. The sum is $1 + 2 + 2 = 5$.) This allow us to know how many nodes (or tasks) there is for a path with a given number of city. When the cutoff of .i.e 8 is reached, we can cut a subtree of all subpath of 8 cities. In this case, it would increment the counter of `SUBTREE_NODES_COUNT_BY_TREE_HEIGHT[8] = 13700` elements with this call `_tasks_done.fetch_add(13700, std::memory_order_relaxed)`. We can simplify the memory ordering constraints from sequential consistency to `relaxed` because this is related to any other variable that would need to be changed in relation.

PriorityStack

This is small structure which inherits `TaskCollection`, used in `ParallelTaskRunner`. This is not thread-safe but it is shared between threads. It is given to `Task::split()` to store the sub tasks based on a given task. The priority aspect on this stack is based on a simple idea. If we can pop the city that will give us the shortest path first, we'll reach the end of a good path faster. Then, this would allow us to cut more branches because the shortest distance found at this point is a bit shorter. To build a simple priority queue, we just sorted the insertion and pop at the back, on a simple `std::vector`. The sort criterion is an estimated cost.

TODO: explain the estimated cost formula.

```

void push(Task* t) override {
    TSPParraTask* tp = static_cast<TSPParraTask*>(t);
    // sorted insert
    auto it = std::upper_bound(_tab.begin(), _tab.end(), tp,
        [](TSPParraTask* a, TSPParraTask* b) { return a->get_estimated_cost() < b->get_estimated_cost(); });
    _tab.insert(it, tp);
}

```

Listing 1:

LockFreeQueue

This implementation is available in file `LockFreeQueue.hpp`. This implementation is based on the course, in chapter 5, with memory ordering constraints added. It works with 2 pointers `head` and `tail` that are `std::atomic`, they are pointing on dummy nodes at start. They allow to build a simply-linked list to represent this queue, as a chain of `Node`. Instead of using null pointers to inform about the empty state, we return a boolean value (`false` if empty) and return the `result` by changing the pointer's reference.

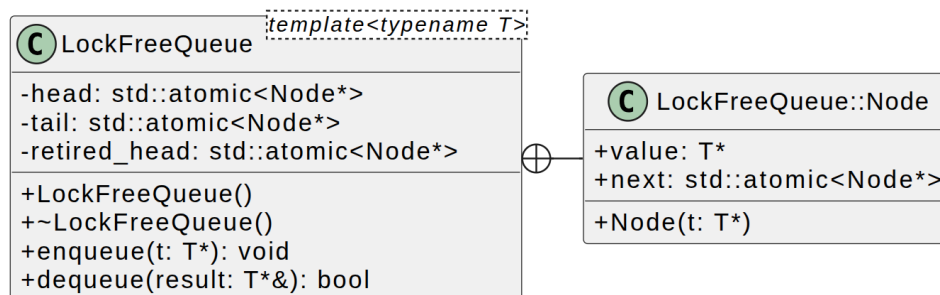


Figure 1: Class diagram of the 2 classes for `LockFreeQueue`

One change from the course's definitions, is the fact that we needed to differ the freeing of removed nodes. We implemented this behavior in `dequeue` with another internal linked list pointed by `retired_head`. Once a node is the result has been read from the first node, we removed it from the queue, we migrate it in "the retire list", which is freed in the destructor.

This queue was first used to store the list of shared tasks in baseline TODO.

Work-stealing deque

This part is developed in `wsd.hpp` and integrated in `parallel_work.hpp`.

The structure presented in the paper [1], is a circular and growable buffer, that is used a double ended queue. One side of the queue (the bottom), each thread will be able to push and pop some tasks. As the other end (the top), some other threads might come to steal some tasks, if their own queue is empty.

As visible in Figure 2 `CircularArray` is the internal contiguous structure dynamically allocated, that can double its capacity when the buffer is full. It is using 2 indexes `bottom` and `top`, which are monotonic values (they always increment and never decrement). These indexes are used modulo the size of the buffer to avoid overflows and enable this circular style. Tasks are represented here with a number, to identify the portion of the buffer that is considered to be used.

The `CircularArray` class is used by `CircularWSDeque`.

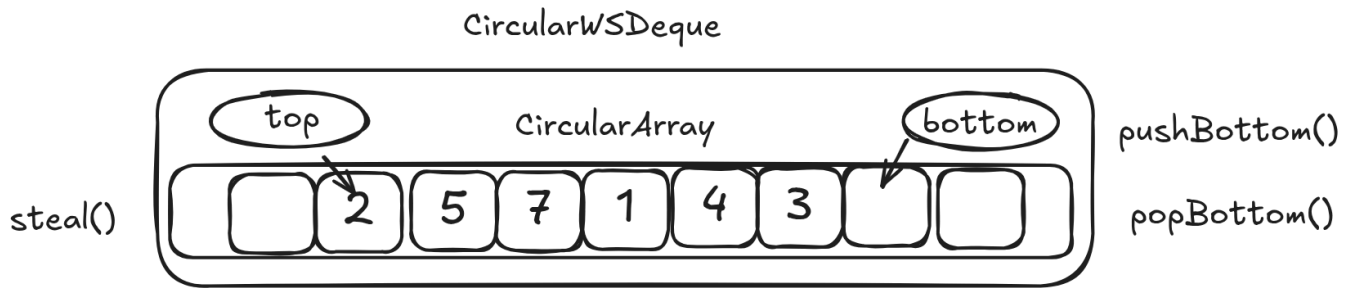


Figure 2: The abstract view of the 2 structures `CircularArray` and `CircularWSDeque`

We are always sure that `top` is lower or equal to `bottom`. `top` is pointing on the top node but `bottom` is pointing on the next cell to be filled by a push on the bottom side. This is leaving an unused cell all the time. The steal on one side `steal()` and the `pushBottom()` and `popBottom()` working on the other side, allow to reduce frequency of threads wanting the work on the same elements. The `pushBottom` and `popBottom` can only be called by the thread owner of the queue. The `steal` method can be called by any thread. The only concurrency problem we have, is when the queue is empty or has one element, the `top` could be changed both from `steal` from other threads and by `popBottom`. This is why this index needs to be protected by a CAS (Compare And Swap).

As visible in Figure 3, all attributes that do change are atomic (all of them, except the size and `stored_size` which are constant). Both classes are generic to make it possible to reuse it with other kind of elements outside of the `Task` interface.

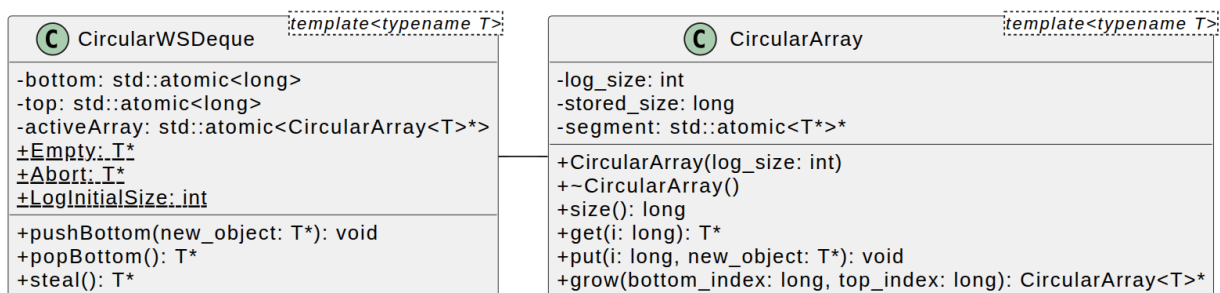


Figure 3: The 2 classes signatures used in the work-stealing implementation

To be able to mark the answer as "empty" (pop and steal) or "abort" (steal), we defined 3 pointers that are markers for such states. To avoid the overhead of a structure with a enum field to discriminate the field, we just defined the empty state as a null pointer. We also defined the abort state as an arbitrary address on the stack, that would never be chosen by a dynamic allocation. With this little hack visible in Listing 2, we can return 8 bytes pointers, instead of 9 or more bytes with a wrapper struct.

```

template<typename T>
class CircularWSDeque {
public:
    static T* Empty;
    static T* Abort;
    // ...
}
static long a = 1;
template<typename T>
T* CircularWSDeque<T>::Empty = nullptr;
template<typename T>
T* CircularWSDeque<T>::Abort = reinterpret_cast<T*>(&a);

```

Listing 2: The initialisation of Empty and Abort static markers pointers in `CircularWSDeque`

The initial size of the array has been defined to 32 in

`int CircularWSDeque<T>::LogInitialSize = 5;` (because 2^5). We didn't measure the cost of growing the arrays, but this might be something to optimize. This could be also adapted dynamically based on the number of cities.

Memory ordering challenges

One major challenge of this implementation was the memory ordering which gave us some headache. The conversion from the Java code in the paper was not as easy as it seems. First we didn't had `std::atomic` around `bottom`, which seems to causes issues where it could not warranty the order of operations before its change. As a small example, the code in Listing 3 is a good example where the CPU reordering of instructions could really be bad. If we consider a scenario where the queue is empty, this snippet is pushing a `new_object` but the CPU reorder the instructions and save `bottom` before the object is really saved in `activeArray` ... This means that an concurrent thread using `steal` on the same queue could take the element without its initialisation to be done.

```

activeArray->put(b, new_object);
bottom = b + 1;

```

Listing 3: An old code example where `bottom` was not `std::atomic`, inside `pushBottom`

To fix the previous issue, we need to make sure instructions cannot be reordered in the wrong way. This is especially the case with lock-free algorithms, where the precise order is giving us very specific coherence or concurrency warranties! In the fixed Listing 4, we now have an `std::atomic` where we can call `store` with the release order, to make sure no instruction done before the store can be seen by other threads after store effect.

```

activeArray->put(b, new_object);
bottom.store(b + 1, std::memory_order_release);

```

Listing 4: An extract of the current `pushBottom` code, where `bottom` is `std::atomic` and we use memory ordering mode



Listing 5:



Listing 6:

Integration

As work-stealing deque is useless, how can we integrate it in the rest of the code ? We need one deque per thread and we need to define 2 strategies: how to init the deques and where to steal work ? First, we implemented a basic way to init the deques. The first deque gets the root task, and all threads are going to come steal their first task into this first deque. It will be filled by subtasks of the root task, splitted by the first thread.

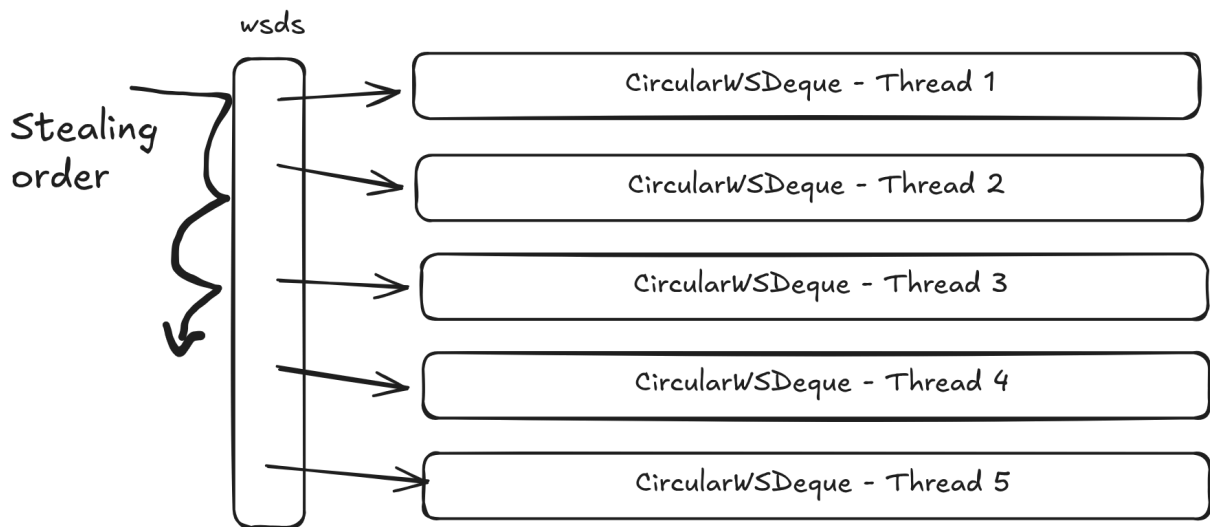


Figure 4: Each thread has it's own work-stealing deque, each thread will start stealing at thread 1 and following

In `ParallelTaskRunner`, the vector of pointer to the deques (`wsds` in the schema) as defined as `std::vector<std::unique_ptr<CircularWSDeque<Task>>> _wsds;` .

This is clearly not the best way to init the deques, as most threads will need to steal thread 1 at the same time, losing some time at start. We didn't have the chance to try it but we would definitely make sure all threads have at least one starting task, to avoid stealing at start. We only developed one level splitting of the root task, which gained us in a hand-made test 3s from 16s in total. We think that the ideal solution is to split the root task recursively in breadth, until we have enough tasks to cover all threads. We would push this list one by one to a different deques each time to ensure proper repartition. Then, it would make sense to have the first thread to steal to be the next one (by attributed thread id) and not the first thread, to avoid contention.

Memory model

Our memory management strategy is as following:

- all tasks are dynamically allocated
- all tasks are deallocated only when it has been managed (splitted in subtasks or solved)
- allocation and deallocation are not using `new` and `delete` all the time, a shared pool of heap memory zones is managed (see `_free_list` below)

The static `thread_local LockFreeQueue<Task>* _free_list;` attribute of `TSPParraTask` is used by `reusealloc` and `reusefree` . A shared list (with only `static`, not `thread_local`) had the first advantage of consuming a lower amount of total memory. But it became a central point of contention. This is why we added `thread_local`, so thread has its own list and doesn't wait on

other threads. Now that only a single thread is using each list, we could have switched back a non thread-safe stack or queue implementation to avoid CAS operations.

The work-stealing pointer to the circular buffer `activeArray` is not cleaned up after a `grow()` because other stealers might still reference the old pointer. We knowingly leak this memory by not calling `delete`. The paper describes a way to reuse this memory but this didn't seem to be an issue, especially with the 126GB of the server. The leaked amount of memory is almost linear to the number of threads (considering there are an almost fixed number of reallocations of and no shrinking).

Crashes

We had a very hard time debugging and thus we spend a significant amount of time trying to fix numerous kind of crashes (use-after-free, segfault of task pointer, out of range on bitset, pop on empty path...). We still have segfaults or some exceptions throwing for cities ≥ 15 . These issues have significantly slowed us down and made it impossible to measure time for more than 16 cities on the server.

```
Thread 247 "tsp" received signal SIGSEGV, Segmentation fault.
(gdb) bt
#0  TSPParraTask::split (this=<optimized out>, collection=<optimized out>) at
parallel_work.hpp:270
#1  TSPParraTask::split (this=0x7ffe500b1e40, collection=0x7ffdf9779dc0) at
parallel_work.hpp:262
#2  0x000000000040287d in ParallelTaskRunner::recurse
(this=this@entry=0x7ffffffffffd0b0, t=0x7ffe500b1e40, tid=tid@entry=245)
    at parallel_work.hpp:364
#3  0x0000000000402d4d in ParallelTaskRunner::worker (this=0x7ffffffffffd0b0, tid=245)
    at parallel_work.hpp:434
#4  0x00007ffff7c4e3e4 in execute_native_thread_routine () from /lib64/libstdc+
+.so.6
#5  0x00007ffff7a53464 in start_thread () from /lib64/libc.so.6
#6  0x00007ffff7ad65ac in __clone3 () from /lib64/libc.so.6
```

Listing 7: Example of GDB backtrace, which is not very helpful in itself

Some issues are caused by duplication of tasks. A task must be only managed by one thread, and because of issues with the Work-stealing deque implementation, some tasks are stolen or stolen+popBottom twice or even three times. We have discovered that by asking ChatGPT to write GoogleTest for us (see `wsd_tests.cpp`) but we just could figure out how to fix our structures. Some issues might be related to memory ordering again.

By the repetition of the last benchmarks, it seems we were able to fix some of them along the way and we restarted them enough to have some measures to discuss.

Measures

Our program has the following arguments.

```
> ./tsp
Usage: ./tsp <file.tsp> [nb cities] [nb threads] [cutoff]
```

We run most of our benchmarks with `hyperfine`, to make sure small we can take an average of several runs. In practice, this is taking too much time with > 15 cities, so we reduce the maximum of executions count.

```
> hyperfine './tsp dj38.tsp 12 50 4'
Benchmark 1: ./tsp dj38.tsp 12 50 4
  Time (mean ± σ):      745.4 ms ±  77.3 ms
  Range (min ... max):  640.8 ms ... 861.4 ms 10 runs
```

Benchmarking system

```
> uv run bench.py init
Machine ID not found, please enter an ID for your machine: srv2
Saved 'srv2' as Machine ID.
No configuration found. Let's set it up.

Enter cities counters (like 5,10,15): 5,10,13
Enter threads counters: 256
Enter cutoff values: 0,1,2,3,4,5,6,7,8,9,10,11,12,13

Configuration saved to bench/configs/srv2.json
```

This is an example of the configuration file generated.

```
{
  "machine_id": "srv2",
  "cities": [ 5, 10, 13 ],
  "threads": [ 256 ],
  "cutoff": [ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13 ]
}
```

Our system `bench.py` allowed us to compare different versions of our program and compare between execution with the previous results. More features are described in `bench/README.md` if needed.

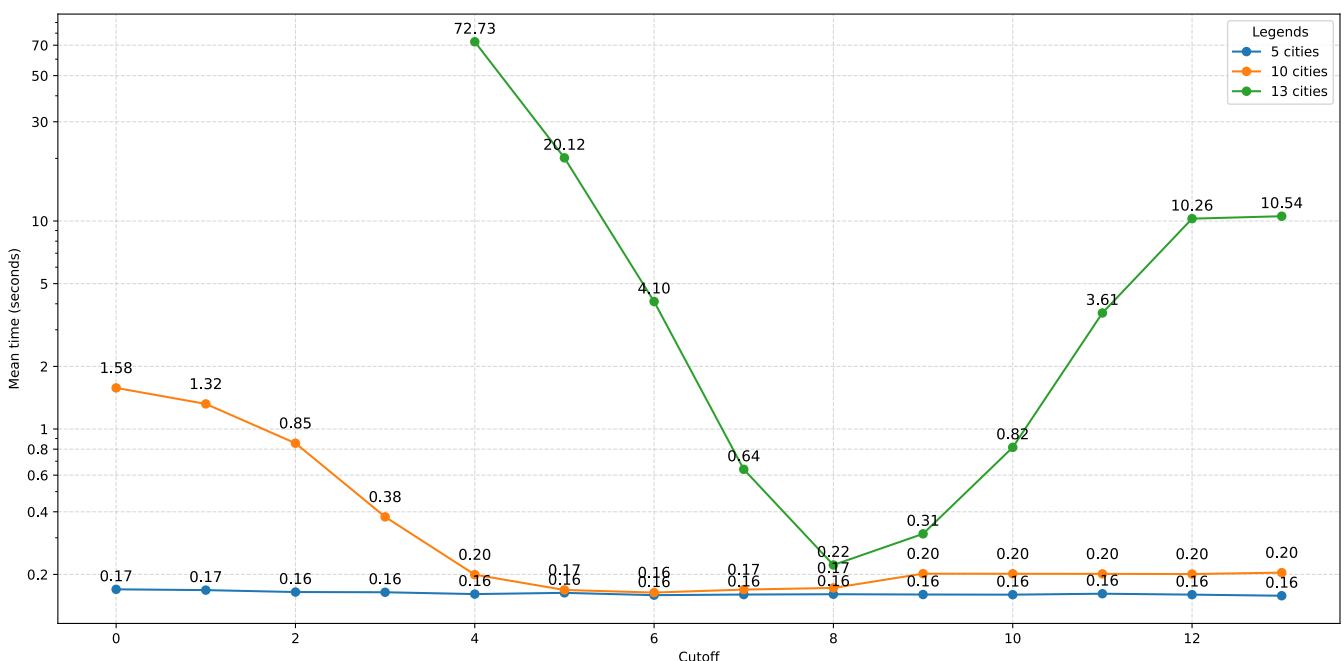


Figure 5: Cutoff analysis - 256 threads - on server

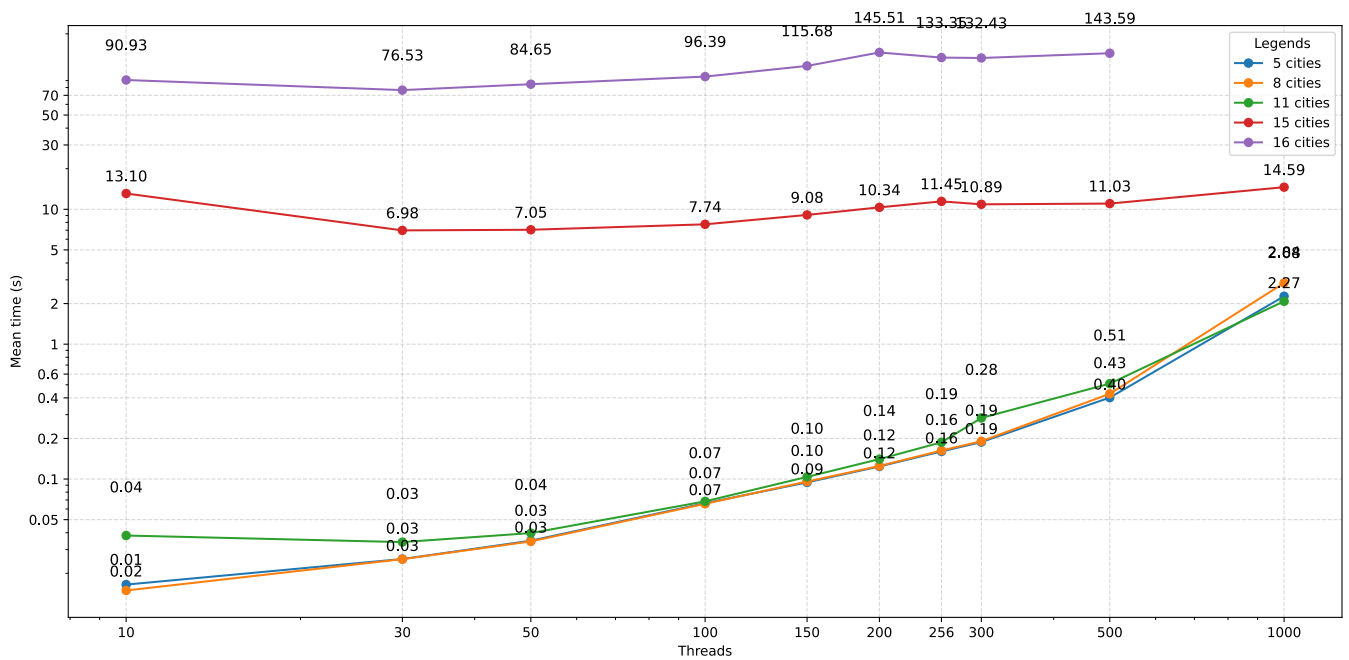


Figure 6: Threads analysis with optimal cutoff at 8 - on server

After this first analysis, we decided the best default cutoff was **8** and the numbers of threads to continue was **30**. We also continued to measure with **256** as the goal was to have this number of threads to be optimal.

TODO

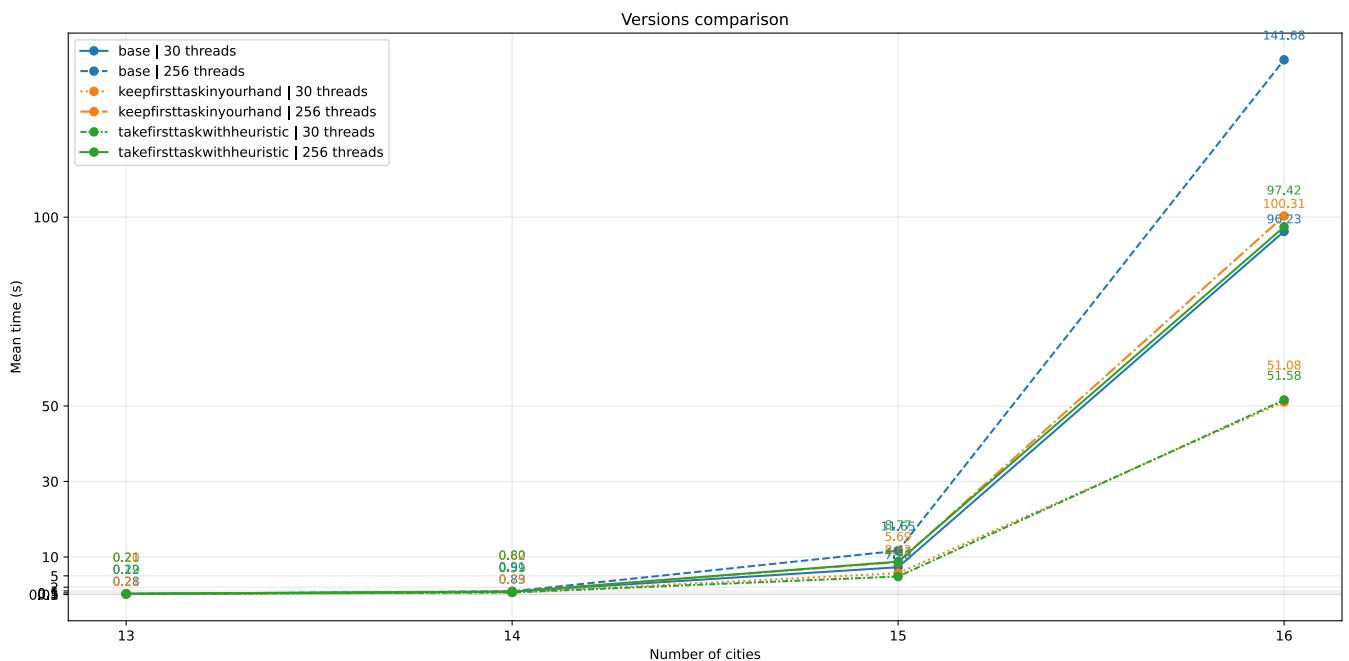


Figure 7: Optimisations results - on server

Speedup

TODO: run sur le serveur le temps de Direct sur 15, 16 et 17 villes.

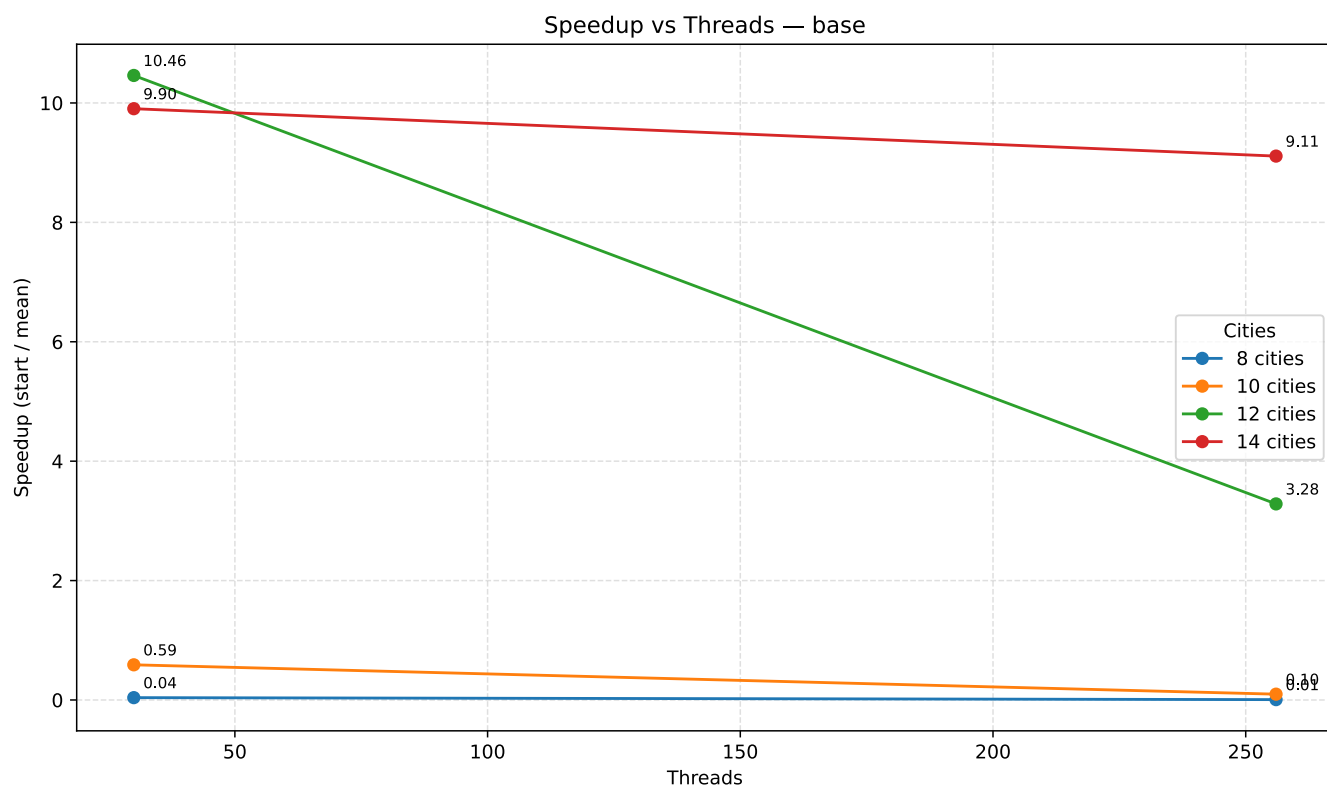


Figure 8: TODO 1

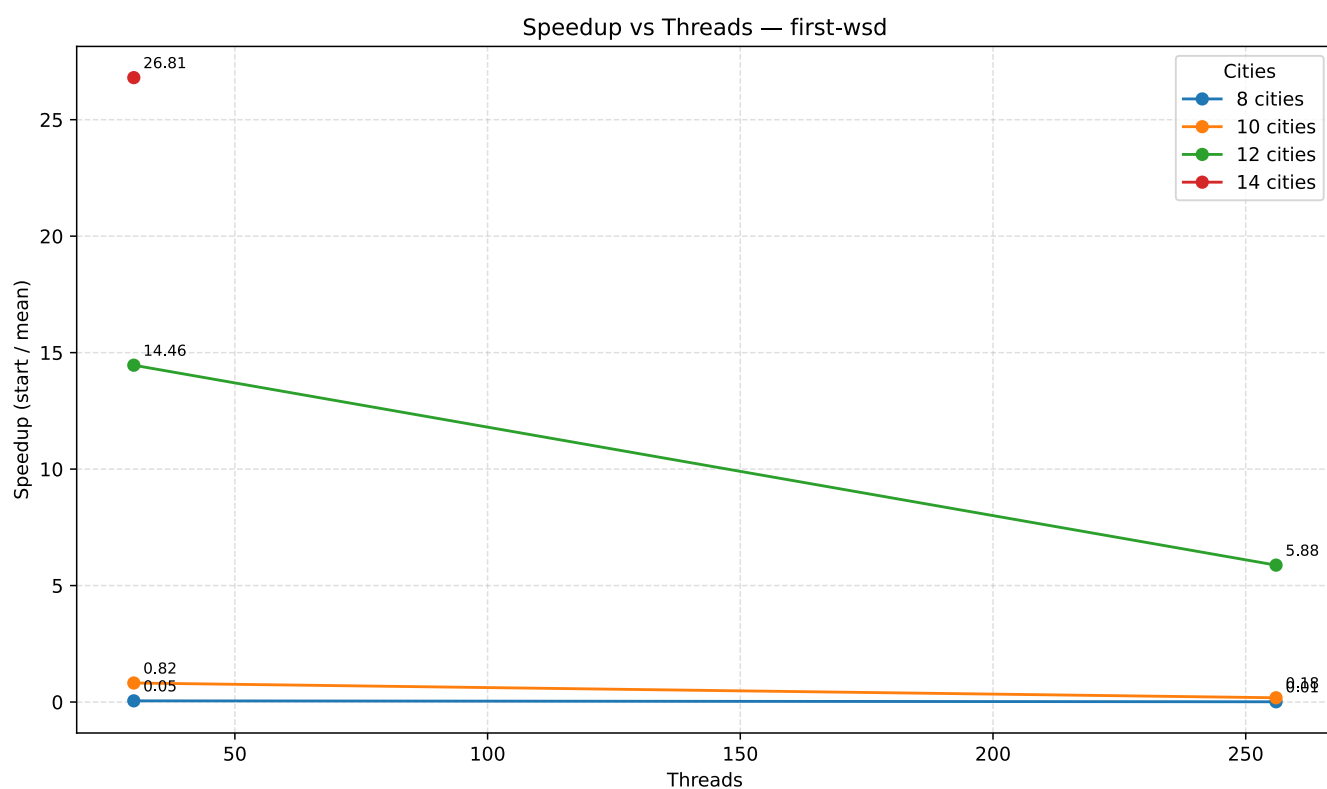


Figure 9: TODO 1

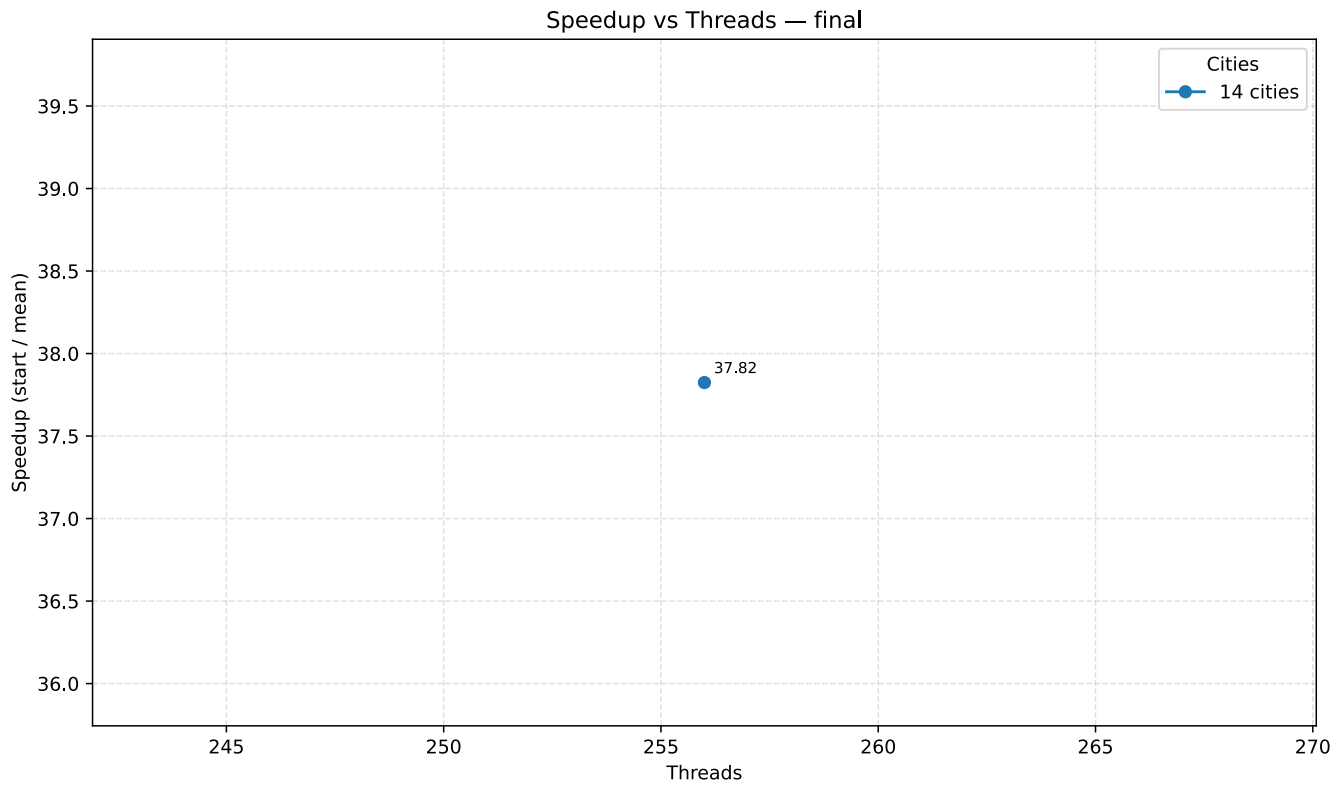


Figure 10: TODO 1

Note: to avoid measuring ourself, we took the values from Rafael and Gaspard for the Direct approach (starting code).

TODO: which baseline has used the LockFreeQueue ?

Efficiency

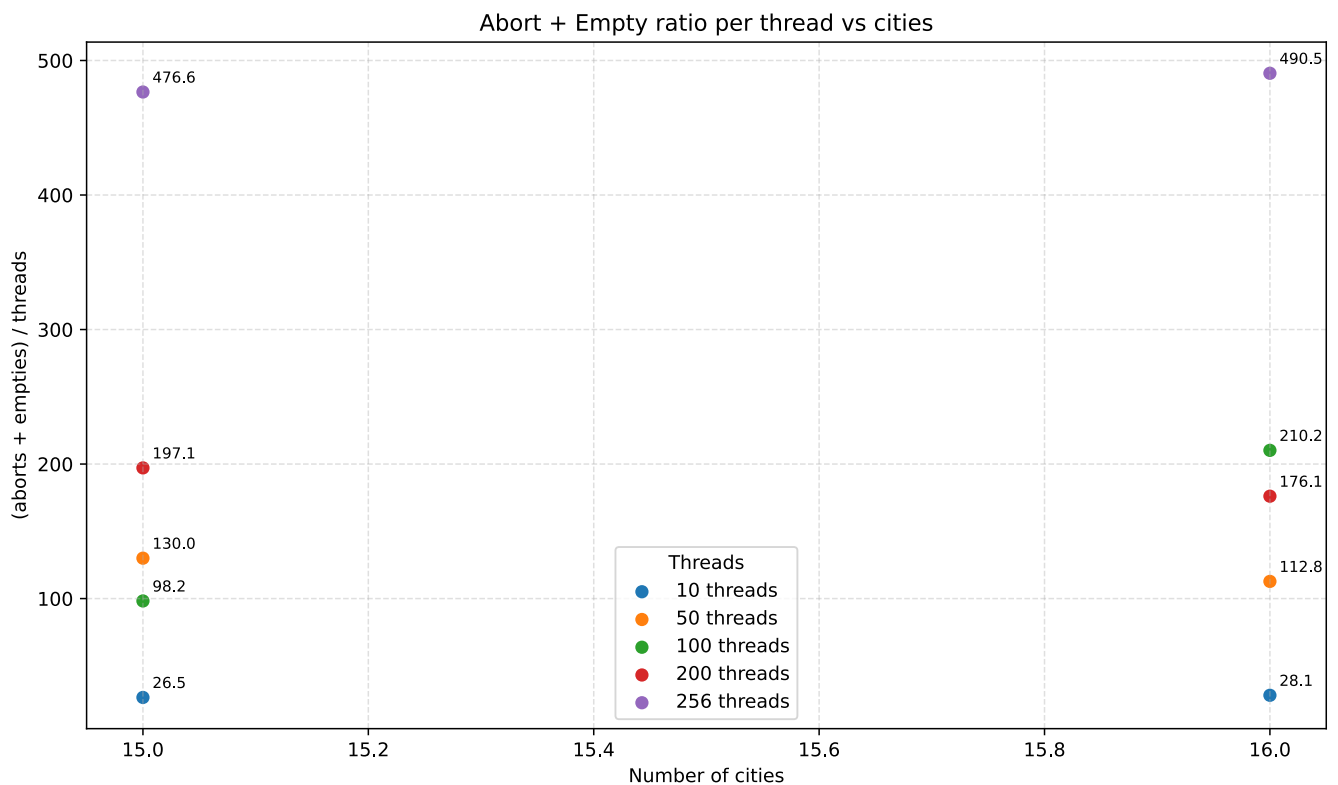


Figure 11: TODO 1

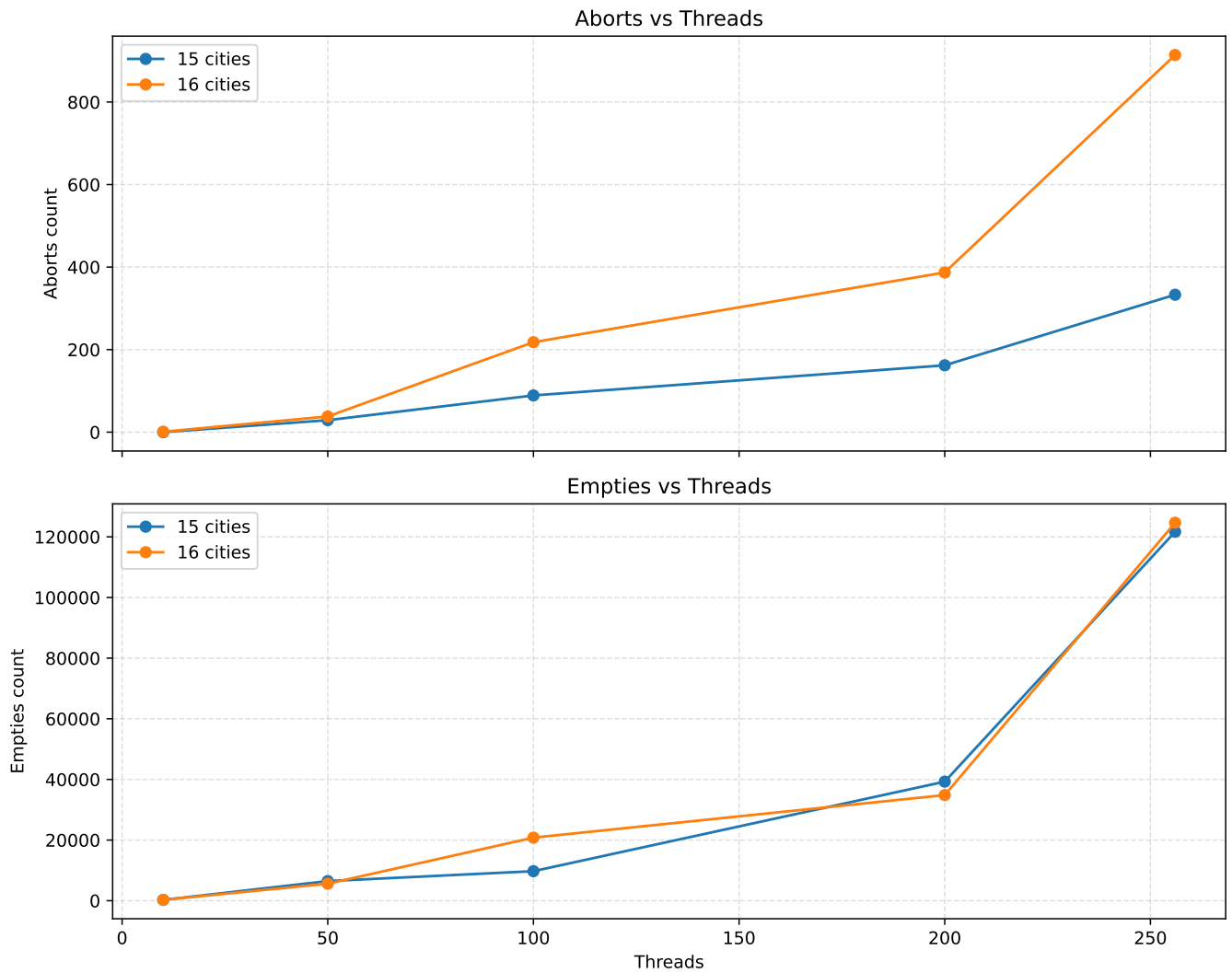


Figure 12: TODO 2

Perspectives

Conclusion

Bibliography

- [1] D. C. et al, "Dynamic circular work-stealing deque." Accessed: Jan. 05, 2026. [Online]. Available: <https://dl.acm.org/doi/10.1145/1073970.1073974>